

ZoumMax

A Decoupled, Depth-Bounded MCTS Variant for
Simultaneous-Move, General-Sum Games

Zouhair Ouddach

Abstract

Monte-Carlo Tree Search is usually presented as a single algorithm, but it is better understood as a family: a selection policy, a tree topology, a simulation policy and a backpropagation rule, each of which can be replaced independently. **ZoumMax** is a variant that replaces three of the four. It abandons the single game tree in favour of *one decoupled search tree per agent*, all descended in lockstep so that depth k in every tree denotes the same simulated turn; it replaces the Monte-Carlo simulation phase with a bounded-depth heuristic evaluation; and it replaces scalar backpropagation with a reward *vector*, one component per agent, each routed only into its own tree. The result is an anytime search for simultaneous-move, N -agent, general-sum domains whose per-agent branching factor is b rather than b^N , and whose node identity is an action prefix rather than a state — making it an open-loop method that is indifferent to stochastic transitions.

1 Where canonical MCTS assumes too much

The canonical formulation — UCT, in the sense of Kocsis and Szepesvári — rests on a set of structural assumptions that are rarely stated because in Go and Hex they are simply true. A node corresponds to a state. Agents act in strict alternation, so each node belongs to exactly one agent, who controls the branching there. Rewards are zero-sum, so a single scalar can be backpropagated and negated along the way. Terminal outcomes are cheaply reachable by random play, which makes the simulation phase informative.

Every one of these assumptions fails in the class of problems ZoumMax targets: real-time, multi-agent, continuous-control domains in which all agents commit to an action within the same tick, observe nothing of each other’s choice beforehand, and are scored by separate and not necessarily opposed objectives.

The naive repair is to keep one tree and branch on the *joint* action. This is correct but unusable: N agents with b discretised actions each produce b^N edges per level, so a node needs $\Theta(b^N)$ visits before its child statistics mean anything, and the visit budget is exhausted at depth one or two. It also hands the search a hidden advantage: within a joint-action node, the tree implicitly knows the opponent’s choice while selecting its own, which is exactly the information a simultaneous-move game withholds.

The MCTS literature offers two families of repair. *Decoupled UCT* keeps one tree over joint states, but stores per-agent statistics at each node and lets each agent run its own bandit over its own action set, recombining the selected actions into a joint move. *Regret-based simultaneous-move MCTS* replaces UCB at those nodes with Exp3 or regret matching, which buys convergence to a Nash equilibrium in the two-player zero-sum case at the cost of much slower practical play. ZoumMax sits beside these and takes the decoupling one step further.

2 The variant: decoupled trees, lockstep descent

ZoumMax gives every agent its own tree, and those trees never touch. A node in agent p ’s tree is identified by the sequence of p ’s *own actions* that reaches it — nothing else. It does not encode the state, and it

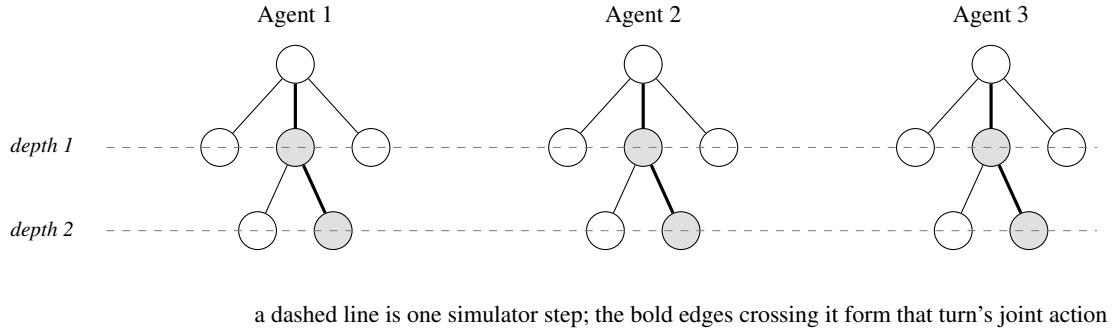
does not encode what the other agents did. Agent p 's tree therefore has branching factor b at every level regardless of N .

An iteration descends all N trees simultaneously, level by level. At depth k , each agent's bandit selects one action inside its own tree; the N selected actions are assembled into a joint action and passed to the simulator, which advances the simulated world one turn. This repeats to a fixed horizon D , at which point an evaluation function scores the reached state once per agent, producing a reward vector $\mathbf{r} = (r_1, \dots, r_N)$. Component r_p is backpropagated up agent p 's traversed path and nowhere else.

Two consequences follow, and they are the substance of the variant.

Interaction is carried by the simulator, not the topology. No agent models any other. The effect of the opponents is felt only as variance in the rewards that flow back into one's own nodes. A node in agent 1's tree aggregates over the distribution of opponent lines that were sampled alongside it, so it is estimating the value of an action prefix *against the opponents' current search-induced policies*, which drift as the search proceeds. The trees stay mutually meaningful because they are always descended together: level k everywhere is turn k of the same hypothetical future.

The search is open-loop. Because node identity is an action prefix and states are never stored, the same node can be reached by many distinct states. This is a deliberate state aggregation. It costs fidelity — two genuinely different positions may share statistics — and it buys three things: memory that is linear in iterations, immunity to stochastic transitions and hidden state (which merely add variance rather than requiring chance nodes), and a tree that is a flat array of small fixed-size records rather than a graph of heavyweight state copies.



3 What ZoomMax changes, phase by phase

Phase	Canonical UCT	ZoomMax
Topology	one tree, nodes = states	N trees, nodes = own action prefixes
Selection	UCB1 on a scalar value	UCB1 on per-node normalised value
Expansion	explicit phase, one node/iter.	folded into selection (untried child first)
Simulation	random playout to terminal	none; heuristic evaluation at depth D
Backpropagation	scalar, negated by ply	vector $\mathbf{r}$, component p into tree p

The selection rule is UCB1 applied to rewards renormalised against the siblings actually observed:

$$a^* = \arg \max_{i \in \text{children}} \left[\hat{v}_i + K \sqrt{\frac{\ln n_{\text{parent}}}{n_i}} \right], \quad \hat{v}_i = \frac{S_i/n_i - m_{\text{parent}}}{M_{\text{parent}} - m_{\text{parent}}}$$

with m and M the running minimum and maximum of the children's mean rewards. Any child with $n_i = 0$ is taken immediately, which is why no separate expansion phase is needed.

This renormalisation is not cosmetic. UCB1's regret bound is stated for rewards in $[0, 1]$, and the exploration term is calibrated to that scale. A heuristic evaluator, unlike a win/loss playout, returns values on an arbitrary

and depth-dependent scale: raw scores in the thousands make the exploration bonus numerical noise and collapse the search into a greedy descent, while scores in the thousandths make the bonus dominate and reduce the search to uniform sampling. Normalising locally, per node, restores the scale the bound assumes and lets a single constant $K \in [0.6, 1.5]$ behave sensibly at every depth. It is the same motivation as the value normalisation used in modern MCTS implementations for unbounded reward domains.

Dropping the simulation phase is the other substantive change. Random playout is informative when random play reaches a terminal outcome whose value is unambiguous; in a continuous-control domain with a long horizon it is close to pure noise, and it consumes the simulator budget that the tree itself could have spent. ZoomMax therefore makes the tree depth *be* the lookahead: every iteration performs exactly D simulator steps, grows each tree by exactly one node, and terminates in a heuristic evaluation. In MCTS terms it is a depth-bounded, evaluation-truncated search — the same move that turned MCTS into the tree component of AlphaZero-style agents, minus the learned value function.

4 The algorithm

Algorithm 1 ZoomMax — one anytime search from the current state

Require: simulator STEP, evaluator EVAL, horizon D , agents $1..N$, budget T

```

1: ROOT[ $p$ ]  $\leftarrow$  fresh node, for each agent  $p$ 
2: while elapsed time  $< T$  do
3:    $s \leftarrow$  copy of the true current state
4:   CUR[ $p$ ]  $\leftarrow$  ROOT[ $p$ ], PATH[ $p$ ]  $\leftarrow \emptyset$  for each agent  $p$ 
5:   for  $k \leftarrow 1$  to  $D$  do
6:     for all agents  $p$  do
7:        $a_p \leftarrow \text{SELECTUCB}(\text{CUR}[p])$   $\triangleright$  untried child first, else the arg max above
8:       CUR[ $p$ ]  $\leftarrow$  child  $a_p$ , creating it if absent; append it to PATH[ $p$ ]
9:     end for
10:     $s \leftarrow \text{STEP}(s, \langle a_1, \dots, a_N \rangle)$   $\triangleright$  one simultaneous tick
11:    if  $s$  is terminal then break
12:  end for
13:   $\mathbf{r} \leftarrow (\text{EVAL}(s, 1), \dots, \text{EVAL}(s, N))$   $\triangleright$  no rollout; evaluation is the leaf value
14:  for all agents  $p$ , for all  $u \in \text{PATH}[p]$  do
15:     $u.n \leftarrow u.n + 1$ ;  $u.S \leftarrow u.S + r_p$ ; refresh the parent's  $(m, M)$ 
16:  end for
17: end while
18: return  $\arg \max_i n_i$  over the children of ROOT[me]
```

The recommended root decision is the most-visited child rather than the highest-valued one, the standard robust-child criterion: under UCB1 the visit distribution is the quantity that concentrates, while a mean over four samples can outrank a mean over nine hundred by luck alone.

5 Theoretical position and known weaknesses

ZoomMax should not be advertised as an equilibrium solver. Deterministic UCB selection at simultaneous-move decision points is known not to converge to a Nash equilibrium in general — the classic counterexample is a biased matching-pennies game, where any deterministic policy is exploitable and the mixed equilibrium is unreachable. Regret-matching or Exp3 selection recovers that guarantee in the zero-sum two-player case; ZoomMax deliberately trades it for sample efficiency, on the pragmatic grounds that in a large continuous domain a strong non-equilibrium policy computed in 50 ms beats a provably-convergent one that has not converged.

Two failure modes follow directly from the design, and both are useful to recognise in a running agent:

Self-confirming lines. Because the trees are descended together and each agent’s bandit concentrates on its own best line, the reward attached to my node is increasingly conditioned on opponent behaviour that my search itself induced. The search can agree with itself. Larger K , more iterations, and occasional forced exploration all dilute the effect.

Aggregation error. Two states reached by the same action prefix share a node. When the domain has strong path-dependence not captured by one’s own actions, the aggregated estimate is biased. This is the price of open-loop search, and the standard mitigations — shorter horizons, richer action encodings — apply.

Finally, the horizon and the action set matter more than K . The action set must be discretised to roughly six to nine representative moves: fewer and the policy cannot express itself, more and no node accumulates enough visits for UCB1 to say anything. The horizon D trades reach against statistical confidence, and is chosen empirically — increase it while the iteration count per decision stays in the thousands, and stop when it does not.

6 Summary

ZoumMax is best read as a point in MCTS design space rather than a new algorithm: decouple the tree per agent instead of per node, index nodes by action prefix instead of state, truncate at a fixed depth with a heuristic instead of rolling out, and backpropagate a reward vector instead of a negated scalar. Each substitution is independently known; the combination is what makes the method viable for N -agent, general-sum, simultaneous-move control under a hard millisecond budget.